ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 20

Bases de datos con objetos

- La estructura del proyecto
- La conexión: una clase y una sola conexión
- La entidad: un curso que sabe cosas
- El repositorio: el único que habla SQL
- Una excepción propia para el borrado
- La sesión, también en una clase
- Las páginas, ahora más cortas
- Antes y después



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

¿Os acordáis del proyecto CRUD de la clase 15? Funcionaba de maravilla... con arrays y funciones sueltas. Desde entonces habéis aprendido clases, herencia, interfaces, excepciones propias, namespaces y Composer. Hoy vamos a **rehacer el mismo proyecto con todo eso**.

La aplicación hará **exactamente lo mismo** que en la clase 15. Lo que cambia es cómo está organizada por dentro. Y esa organización tiene nombre propio, porque la vais a encontrar en cualquier proyecto profesional y en Laravel: el patrón **Repository**.

•
👌 Cómo vamos a trabajar



1
🤔 Lo que vamos a cambiar



2
📘 La estructura del proyecto



3
👍 La conexión: una clase y una sola conexión



4
🔍 La entidad: un curso que sabe cosas



5
🚀 El repositorio: el único que habla SQL



6
🐞 Una excepción propia para el borrado



7
⚠️ La sesión, también en una clase



8
👌 Las páginas, ahora más cortas



9
😮 Antes y después



10
🚀 Ejercicios



11
👋 Lo que tenéis que llevaros de hoy





CLASE 20

👌 Cómo vamos a trabajar

1. En Laragon, cread la carpeta `academia-poo` en la raíz y copiad dentro el contenido de `proyecto/`.
2. Si no la tenéis de la clase 15, cread la base de datos ejecutando `base-de-datos.sql` en HeidiSQL. Es la misma: `academia_crud`.
3. Abrid la **Terminal** de Laragon, entrad en la carpeta y ejecutad `composer install` para generar la autocarga.
4. Pulsad **Recargar** y abrid `http://academia-poo.test`. El usuario sigue siendo `admin@academia.test` / `secreto123`.



💡 Este proyecto no instala ninguna librería: usamos Composer solo para la **autocarga** de nuestras clases. Sin `composer install`, veréis el error de la clase 19: *Failed opening required 'vendor/autoload.php'*.



CLASE 20 · PUNTO 1

🤔 Lo que vamos a cambiar

En la clase 15	En la clase 20
Una función `conectar()` que crea una conexión nueva cada vez	Una clase `Conexion` que reutiliza **una sola** conexión
Cada curso es un **array**: `$curso["titulo"]`	Cada curso es un **objeto** `Curso`: `$curso->titulo`
`validarCurso($curso)` suelta	El propio curso sabe si es válido: `$curso->errores()`
Funciones de SQL sueltas en `includes/cursos.php`	Una clase `CursoRepositorio` con todas las consultas
`crearCurso()` y `actualizarCurso()`	Un único `guardar()`: si el curso no tiene id, lo crea; si lo tiene, lo actualiza
Capturar el error 1451 en `borrar.php`	El repositorio lanza una `CursoConAlumnosException`
Funciones de sesión sueltas	Una clase `Sesion`
Cuatro `require_once` en `inicio.php`	`vendor/autoload.php` y PSR-4



CLASE 20 · PUNTO 2

📘 La estructura del proyecto

Esquema

```
academia-poo/
├── composer.json              ← autocarga PSR-4 (App\ → src/) y src/funciones.php
├── config/config.php          ← constantes de configuración
├── includes/                  ← inicio.php, cabecera, pie y formulario
├── src/                       ← App\
│   ├── BaseDatos/
│   │   └── Conexion.php       ← App\BaseDatos\Conexion
│   ├── Modelos/
│   │   ├── Curso.php          ← la entidad Curso
│   │   └── Usuario.php
│   ├── Repositorios/
│   │   ├── CursoRepositorio.php
│   │   └── UsuarioRepositorio.php
│   ├── Excepciones/
│   │   └── CursoConAlumnosException.php
│   ├── Web/
│   │   └── Sesion.php
│   └── funciones.php          ← e(), euros() y redirigir()
├── index.php, crear.php, editar.php, borrar.php, login.php, logout.php
└── estilos.css
```

Fijaos en `src/funciones.php`. No es una clase: son las tres funciones de siempre. Composer también sabe cargar archivos así con la opción **`files`** [1]:

datos.json — php-paso-a-paso/clase20 — Visual Studio Code

datos.json

```json
{
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        },
        "files": [
            "src/funciones.php"
        ]
    }
}
```

📄 Archivo: `composer.json`



CLASE 20 · PUNTO 3

👍 La conexión: una clase y una sola conexión

En la clase 15, cada vez que llamabais a `conectar()` se abría una conexión **nueva** con MySQL. Si una página necesita dos repositorios, son dos conexiones, y cada una tarda un poquito. La clase `Conexion` guarda la primera en una propiedad **estática** y la reutiliza:

Conexion.php — php-paso-a-paso/clase20 — Visual Studio Code

Conexion.php

```php
<?php
declare(strict_types=1);

namespace App\BaseDatos;

use PDO;
use PDOException;

class Conexion
{
    private static ?PDO $pdo = null;

    public static function obtener(): PDO
    {
        if (self::$pdo === null) {
            self::$pdo = self::conectar();
        }
        return self::$pdo;
    }

    private static function conectar(): PDO
    {
        // el mismo DSN y las mismas tres opciones de la clase 14
    }
}
```

La primera vez que alguien pide `Conexion::obtener()`, se conecta; las siguientes, devuelve la misma. Fijaos en el `use PDO;`: en un archivo con namespace, las clases de PHP hay que importarlas (clase 19).

📄 Archivo: `src/BaseDatos/Conexion.php`



💡 Esta idea de «una sola instancia compartida» tiene nombre: es una versión sencilla del patrón ***Singleton***. Lo veréis citado a menudo.



CLASE 20 · PUNTO 4

🔍 La entidad: un curso que sabe cosas

Una **entidad** es una clase que representa una cosa de vuestra aplicación: un curso, un alumno, un pedido. Guarda sus datos y sabe hacer lo que tiene sentido para ella. Lo importante es lo que **no** hace: no sabe nada de bases de datos [2].

Curso.php — php-paso-a-paso/clase20 — Visual Studio Code

Curso.php

```php
<?php
namespace App\Modelos;

class Curso
{
    public const IVA = 0.21;

    public function __construct(
        public ?int $id,        // null mientras el curso no se ha guardado
        public string $titulo,
        public int $horas,
        public float $precio,
        public int $plazas,
    ) {
    }

    public static function desdeFila(array $fila): Curso { /* ... */ }
    public static function desdeFormulario(array $post, ?int $id = null): Curso { /* ... */
}

    public function errores(): array { /* las mismas reglas de validarCurso() */ }
    public function esValido(): bool { return count($this->errores()) === 0; }
    public function precioConIva(): float { /* ... */ }
    public function estaCompleto(): bool { return $this->plazas === 0; }
}
```

Método	Para qué
`Curso::desdeFila($fila)`	Convertir una fila de la base de datos en objeto (lo usa el repositorio)
`Curso::desdeFormulario($_POST)`	Convertir lo que llega del formulario en objeto (lo usan crear y editar)
`$curso->errores()`	Las reglas de validación, dentro del propio curso
`$curso->precioConIva()` y `$curso->estaCompleto()`	Lo que el curso «sabe» de sí mismo

¿Veis por qué el `id` es `?int`? Un curso recién salido del formulario **todavía no tiene id**: se lo dará MySQL al guardarlo. Ese `null` nos va a servir enseguida.

📄 Archivo: `src/Modelos/Curso.php`



Pensad en el **almacén** de una tienda. El dependiente no baja a buscar entre las estanterías: le pide al encargado del almacén «tráeme la camiseta 42» o «guarda esta caja». Solo el encargado sabe dónde está cada cosa.

Un **repositorio** es ese encargado: la única clase que sabe hablar SQL con una tabla. Las páginas le piden y le entregan **objetos**, y no ven ni una consulta [3].

CursoRepositorio.php — php-paso-a-paso/clase20 — Visual Studio Code

CursoRepositorio.php

```php
<?php
namespace App\Repositorios;

use App\Modelos\Curso;
use PDO;

class CursoRepositorio
{
    public function __construct(private PDO $pdo)
    {
    }

    public function contar(string $busqueda = ""): int { /* SELECT COUNT(*) ... */ }

    /** @return Curso[] */
    public function buscar(string $busqueda, int $pagina, int $porPagina): array
    {
        // el mismo SELECT con LIMIT y OFFSET de la clase 15...
        return array_map(fn(array $fila) => Curso::desdeFila($fila), $consulta->fetchAll());
    }

    public function buscarPorId(int $id): ?Curso { /* ... o null si no existe */ }

    public function guardar(Curso $curso): void
    {
        if ($curso->id === null) {
            // INSERT, y le damos al objeto su id nuevo
            $curso->id = (int) $this->pdo->lastInsertId();
        } else {
            // UPDATE ... WHERE id = :id
        }
    }

    public function borrar(int $id): bool { /* DELETE ... */ }
}
```

Tres detalles:

- **El repositorio recibe la conexión en su constructor** (`private PDO $pdo`), no se la inventa. Así, varios repositorios pueden compartir la misma conexión. Se llama **inyección de dependencias** y la veréis mucho en Laravel.
- `guardar()` **decide solo** si es `INSERT` o `UPDATE`, mirando si el curso tiene id. Las páginas ya no tienen que acordarse.
- **El comentario** `/** @return Curso[] */` le dice a VS Code (y a quien lea el código) que el array contiene objetos `Curso`. Así el editor os autocompleta `$curso->` .

📄 Archivos: `src/Repositorios/CursoRepositorio.php` y `src/Repositorios/UsuarioRepositorio.php`



🎯 Esta separación es la idea más importante de la clase: **las entidades no saben de SQL y las páginas no saben de SQL**. Todo el SQL de los cursos vive en un solo archivo.



CLASE 20 · PUNTO 6

🐞 Una excepción propia para el borrado

En la clase 15, `borrar.php` capturaba la `PDOException` y miraba si el código era 1451. Eso es un detalle **de MySQL**, y la página no debería saberlo. Ahora lo resuelve el repositorio, que lo traduce a una excepción con nombre propio (clase 18):

CursoConAlumnosException.php — php-paso-a-paso/clase20 — Visual Studio Code

CursoConAlumnosException.php

```php
<?php
public function borrar(int $id): bool
{
    try {
        $consulta = $this->pdo->prepare("DELETE FROM cursos WHERE id = :id");
        $consulta->execute(["id" => $id]);
        return $consulta->rowCount() === 1;
    } catch (PDOException $e) {
        if (($e->errorInfo[1] ?? 0) === 1451) {
            throw new CursoConAlumnosException($id);
        }
        throw $e;
    }
}
```

Y la página solo tiene que entender **qué** ha pasado, no **cómo** lo dice MySQL:

CursoConAlumnosException.php — php-paso-a-paso/clase20 — Visual Studio Code

CursoConAlumnosException.php

```php
<?php
try {
    $cursosRepo->borrar($id);
} catch (CursoConAlumnosException $e) {
    Sesion::guardarMensaje($e->getMessage(), "error");
}
```

📄 Archivos: `src/Excepciones/CursoConAlumnosException.php` y `borrar.php`



CLASE 20 · PUNTO 7

⚠️ La sesión, también en una clase

Los mensajes *flash*, el token CSRF y el usuario conectado se guardan en `$_SESSION`. En vez de funciones sueltas, ahora están todos juntos en la clase `Sesion`, con métodos estáticos:

Clase 15	Clase 20
`guardarMensaje("...")`	`Sesion::guardarMensaje("...")`
`tokenCsrf()` / `comprobarCsrf($t)`	`Sesion::tokenCsrf()` / `Sesion::comprobarCsrf($t)`
`requerirLogin()`	`Sesion::requerirLogin()`
`$_SESSION["usuario"] = [...]` en `login.php`	`Sesion::entrar($usuario)`

Además, el `Usuario` ya no deja salir el hash de su contraseña: `$usuario->comprobarClave($clave)` hace el `password_verify()` por dentro.

📄 Archivos: `src/Web/Sesion.php` y `src/Modelos/Usuario.php`



CLASE 20 · PUNTO 8

👌 Las páginas, ahora más cortas

Así queda `crear.php`. Leedlo en voz alta: casi es una frase en castellano.

index.php — php-paso-a-paso/clase20 — Visual Studio Code

index.php

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/includes/inicio.php";

use App\BaseDatos\Conexion;
use App\Modelos\Curso;
use App\Repositorios\CursoRepositorio;
use App\Web\Sesion;

$titulo = "Nuevo curso";
$errores = [];
$curso = Curso::vacio();

Sesion::requerirLogin();
$cursosRepo = new CursoRepositorio(Conexion::obtener());
$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";

if ($esEnvio) {
    Sesion::comprobarCsrf($_POST["csrf"] ?? "");
    $curso = Curso::desdeFormulario($_POST);
    $errores = $curso->errores();

    if ($curso->esValido()) {
        $cursosRepo->guardar($curso);
        Sesion::guardarMensaje("Curso «{$curso->titulo}» creado con el id {$curso->id}.");
        redirigir("index.php");
    }
}

require __DIR__ . "/includes/cabecera.php";
?>
```

Fijaos en el mensaje: después de `guardar()`, el objeto `$curso` **ya tiene su id**, porque el repositorio se lo ha puesto. Y en el listado, `$curso->precioConIva()` y `$curso->estaCompleto()` sustituyen a los cálculos que antes hacíamos en la plantilla.

📄 Archivos: `index.php`, `crear.php`, `editar.php`, `borrar.php` y `login.php`

✅ Pruebas para comprobarlo

Son las mismas de la clase 15, y deben dar el mismo resultado: catálogo, buscador, paginación, login, alta con errores, edición precargada, borrado de un curso con alumnos (ahora con el mensaje de la excepción propia), borrado por GET (405) y cierre de sesión.



CLASE 20 · PUNTO 9

😮 Antes y después

¿Ha merecido la pena? Juzgad vosotros:

Si mañana…	Clase 15	Clase 20
Cambia el nombre de una columna	Buscar en `cursos.php` … y en las plantillas que usan `$curso["columna"]`	Solo `CursoRepositorio` y `Curso::desdeFila()`
Cambia una regla de validación	`validarCurso()`	`Curso::errores()`
Queréis mostrar el precio con IVA en otra página	Copiar el cálculo	`$curso->precioConIva()`
Necesitáis los cursos en otra parte (una API, un informe)	Copiar funciones o incluir archivos	`new CursoRepositorio(...)` y listo
Queréis probar sin base de datos	Imposible sin cambiar código	Otro repositorio que cumpla la misma interfaz (ejercicio 03)

Más archivos, sí. Pero cada uno más pequeño, con un solo trabajo y fácil de encontrar. Y lo mejor: esta organización es **la misma** que vais a encontrar en el MVC y en Laravel.



CLASE 20 · PUNTO 10

🚀 Ejercicios

Como en la clase 15, las soluciones se copian **dentro del proyecto**: los archivos de `soluciones/src/` en `src/`, y las páginas en la raíz.

- ☐ **01 · Alumnos.** Una entidad `Alumno` y un `AlumnoRepositorio` con `todosConCurso()` (con `INNER JOIN`). Mostrad el listado en `alumnos.php`. → `Alumno.php`, `AlumnoRepositorio.php` y `alumnos.php`
- ☐ **02 · Ficha del curso.** `ver.php?id=…` con los datos del curso y sus alumnos, usando **los dos** repositorios con **la misma** conexión. Pista: añadid `delCurso()` al `AlumnoRepositorio`. → `ver.php`
- ☐ **03 · Repositorio en memoria.** Cread una interfaz `CursoRepositorioInterface` y un `CursoRepositorioMemoria` que guarde los cursos en un array. Escribid una función que reciba **cualquier** repositorio que cumpla la interfaz. → `CursoRepositorioInterface.php`, `CursoRepositorioMemoria.php` y `prueba-memoria.php`
- ☐ **04 · Registro.** Añadid a `UsuarioRepositorio` un método `crear()` que guarde la contraseña con `password_hash()`, y una página `registro.php`. → `UsuarioRepositorio.php` y `registro.php`



🏆 **Reto:** haced que `CursoRepositorio` implemente también la interfaz del ejercicio 03 (`class CursoRepositorio implements CursoRepositorioInterface`) y cambiad el tipo de los parámetros de vuestras funciones a la interfaz. Ahora podéis cambiar de MySQL a memoria tocando **una sola línea**. Así es como se preparan las pruebas automáticas.



CLASE 20 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`Conexion`	Una sola conexión para toda la petición, guardada en una propiedad estática
Entidad	Una clase con los datos y el comportamiento de una cosa (`Curso`), sin SQL
`desdeFila()` / `desdeFormulario()`	Métodos estáticos que fabrican objetos a partir de arrays
Repositorio	La única clase que habla SQL con una tabla; recibe y devuelve objetos
Inyección de dependencias	El repositorio recibe la conexión en el constructor
`guardar()`	Sin id, `INSERT` ; con id, `UPDATE`
Excepciones propias	El repositorio traduce los errores de MySQL a nombres que se entienden
`files` en Composer	Para cargar archivos de funciones junto a la autocarga PSR-4

📚 Fuentes y documentación oficial

1. Composer: the autoload field (files)
2. Martin Fowler: Catalog of Patterns of Enterprise Application Architecture
3. Martin Fowler: Repository